

Advice

Victoria Iașcevschi, Flavius Petrașciuc & Stefan Bryda

Introduction

This document provides an architectural improvement analysis for the existing JabberPoint system. The goal of this redesign is to improve the maintainability, extensibility, and overall design quality of the application by applying established object-oriented design principles and design patterns.

The original system functions correctly but contains several structural weaknesses, including tight coupling between components, responsibilities mixed across layers, and object creation logic embedded in classes that should not be responsible for it.

To address these issues, several improvements were introduced to the class diagram and system architecture. The redesign applies the following design patterns:

- **Observer Pattern**
- **Factory Method**
- **Command Pattern**

Additionally, several modifications were made to existing classes to improve adherence to clean code principles such as separation of concerns and loose coupling.

The following sections describe the problems identified in the original design, the modifications made, and the benefits of the new architecture.

1. Observer Pattern

Issue

The *Presentation* class had a **direct dependency on *SlideViewerComponent***.

This tightly coupled the **core presentation logic with the UI layer**, making the system difficult to extend with additional viewers.

Changes Made

- Added a new interface: *PresentationObserver*
- Modified *Presentation* to maintain a list of observers.
- *SlideViewerComponent* now implements *PresentationObserver*.
- Removed the direct dependency between *Presentation* and *SlideViewerComponent*.

Why

This change **decouples the model from the UI**.

Now multiple components can observe presentation changes without modifying the *Presentation* class.

2. Factory Pattern

Issue

The XMLAccessor class was directly creating objects such as:

TextItem & BitmapItem

This meant the class needed to know every subclass of *SlideItem*, violating the **Open/Closed** Principle.

Changes Made

- Introduced a new class: *SlideItemFactory*
- *XMLAccessor* now delegates object creation to the factory.
- The factory decides which subclass of *SlideItem* to create.

Why

Centralizing object creation **reduces coupling** and allows new slide item types to be added with minimal changes to the system.

3. Command Pattern

Issue

MenuController and *KeyController* directly called methods in the *Presentation* class. This mixed **UI logic with application logic** and duplicated functionality.

Changes Made

- Introduced a Command interface with an `execute()` method.
- Added command classes such as:
 - `NextSlideCommand`
 - `PreviousSlideCommand`
 - `OpenCommand`
 - `SaveCommand`
 - `ExitCommand`
- Controllers now trigger commands instead of calling presentation methods directly.

Why

Encapsulating actions into command objects improves **separation of concerns** and allows commands to be reused by different UI elements. Before adding new actions required modifying controller logic which violated the Open/Closed Principle.